

TAREFA 4: TRATAMENTO DE DEADLOCK (de 19/12/2024 a 13/01/2025)

Considerando o algoritmo do jantar dos filósofos, implemente o cenário proposto no Pseudocódigo 2 usando *threads* e semáforos.

```
#define N      5          /* número de filósofos */
#define LEFT   (i+N-1)%N  /* número do vizinho à esquerda de i */
#define RIGHT  (i+1)%N    /* número do vizinho à direita de i */
#define THINKING 0        /* o filósofo está pensando */
#define HUNGRY   1         /* o filósofo está tentando pegar garfos */
#define EATING   2         /* o filósofo está comendo */
typedef int semaphore;    /* semáforos são um tipo especial de int */
int state[N];             /* arranjo para controlar o estado de cada um */
semaphore mutex = 1;      /* exclusão mútua para as regiões críticas */
semaphore s[N];           /* um semáforo por filósofo */

void philosopher(int i)    /* i: o número do filósofo, de 0 a N-1 */
{
    while (TRUE) {         /* repete para sempre */
        think();           /* o filósofo está pensando */
        take_forks(i);     /* pega dois garfos ou bloqueia */
        eat();             /* hummm! Espagete! */
        put_forks(i);      /* devolve os dois garfos à mesa */
    }
}

void take_forks(int i)     /* i: o número do filósofo, de 0 a N-1 */
{
    down(&mutex);          /* entra na região crítica */
    state[i] = HUNGRY;     /* registra que o filósofo está faminto */
    test(i);               /* tenta pegar dois garfos */
    up(&mutex);            /* sai da região crítica */
    down(&s[i]);            /* bloqueia se os garfos não foram pegos */
}

void put_forks(i)         /* i: o número do filósofo, de 0 a N-1 */
{
    down(&mutex);          /* entra na região crítica */
    state[i] = THINKING;   /* o filósofo acabou de comer */
    test(LEFT);            /* vê se o vizinho da esquerda pode comer agora */
    test(RIGHT);           /* vê se o vizinho da direita pode comer agora */
    up(&mutex);            /* sai da região crítica */
}

void test(i)              /* i: o número do filósofo, de 0 a N-1 */
{
    if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
        state[i] = EATING;
        up(&s[i]);
    }
}
```

Pseudocódigo 2: Solução algorítmica para tratamento de deadlock - jantar dos filósofos.